CO1 AT-2
Error Analysis Task

Name: - D. Sri Anjaneya **Reg. No.: -** 192425219

Course Name: - Deep Learning **Course Code: -** MLA 0409

1. Error Analysis in Maximum Likelihood Estimation (MLE)

A student builds a binary classifier using Maximum Likelihood Estimation. The likelihood function is incorrectly written as the sum of probabilities instead of the product. The model produces poor parameter estimates even with large data.

Tasks:

a) Identify the conceptual error in the likelihood formulation.

b) Explain how this mistake affects parameter estimation.

c) Suggest the correct mathematical formulation and reasoning.

d) How would using log-likelihood fix computational issues?

Answer:

Error Analysis in Maximum Likelihood Estimation (MLE):

a) Conceptual error in likelihood formulation:

Error:

The student used sum of probabilities:

$$L(\theta) = P(x_1 \mid \theta) + P(x_2 \mid \theta) + \ldots + P(x_n \mid \theta)$$

instead of the correct product of probabilities:

$$L(\theta) = \prod_{i=1}^{n} P(x_i \mid \theta)$$

MLE requires the joint probability of all samples, so multiplication is used.

b) Effect on parameter estimation

- The model learns incorrect parameters because it is not maximizing the true likelihood.
- The estimated values become biased.
- Even with more training data, the model may not improve.
- Classification accuracy decreases.

c) Correct mathematical formulation

The correct likelihood is:

$$L(\theta) = \prod_{i=1}^{n} P(x_i \mid \theta)$$

The best parameters are found by:

$$\hat{\theta} = argmax_{\theta} L(\theta)$$

This selects parameters that make the observed data most likely.

d) Role of log-likelihood

Log-likelihood converts multiplication into addition:

$$\log L(\theta) = \sum_{i=1}^{n} \log P(x_i \mid \theta)$$

Benefits:

- Avoids numerical problems caused by multiplying very small probabilities.
- Makes calculations easier.
- Improves optimization speed.
- Gives the same parameter values as normal likelihood.

Conclusion:

The mistake of using a sum instead of a product makes MLE mathematically incorrect. Using the correct likelihood with log transformation improves accuracy, stability, and computational efficiency.

2. Error Diagnosis in Perceptron Learning

While implementing a Perceptron, a student observes that the model fails to converge even after many iterations on a linearly separable dataset.

Tasks:

a) List possible implementation errors in the learning algorithm.

b) Analyze how incorrect learning rate or weight updates affect convergence.

c) Identify dataset-related issues that may cause this problem.

d) Propose modifications to ensure convergence.

Answer:

a) Possible implementation errors:

- Incorrect weight update formula.
- Updating weights even when prediction is correct.
- Wrong calculation of error:

$$error = y - \hat{y}$$

- Incorrect bias update.
- Not initializing weights properly.
- Wrong activation function implementation.

b) Effect of incorrect learning rate or weight updates:

- Very high learning rate:
  - Causes large weight changes.
  - The model may keep jumping and fail to converge.
- Very low learning rate:
  - Weight changes become too small.
  - Convergence becomes very slow.
- Wrong weight update:

Correct update:

$$w = w + \eta(y - \hat{y})x$$

If this is implemented incorrectly, the decision boundary moves in the wrong direction and convergence fails.

c) Dataset-related issues:

- Dataset may not actually be linearly separable.

- Presence of noisy or incorrect labels.
- Outliers affecting the decision boundary.
- Features may have different scales.
- Insufficient training samples.

d) Modifications to ensure convergence:

- Normalize or standardize input features.
- Choose an appropriate learning rate.
- Verify the weight update rule.
- Remove noisy data points.
- Increase training iterations.
- Use improved algorithms like:
    - Pocket Perceptron
    - Support Vector Machine (SVM) for non-separable data

Conclusion:

Perceptron fails to converge mainly due to incorrect implementation, unsuitable learning rate, or data problems. Correct updates, feature scaling, and proper preprocessing help achieve convergence.

3. Backpropagation Gradient Error Analysis

A Multilayer Perceptron trained using Backpropagation shows increasing loss during training instead of decreasing.

Tasks:

a) Identify possible mathematical or coding errors in gradient computation.

b) Explain the role of activation functions in gradient flow.

c) Analyze how vanishing or exploding gradients affect training.

d) Suggest techniques to fix the issue (e.g., normalization, initialization).

Answer:

a) Possible mathematical or coding errors:

- Incorrect calculation of gradients using wrong derivative formulas.
- Wrong chain rule implementation during backpropagation.
- Incorrect weight update equation.
- Updating weights before completing gradient calculation.
- Mistake in loss function implementation.
- Incorrect learning rate value.
- Wrong initialization of weights.

b) Role of activation functions in gradient flow:

- Activation functions decide how gradients pass through the network.
- Sigmoid/Tanh functions can reduce gradients when values become saturated.
- ReLU helps maintain stronger gradients but can cause dead neurons.
- Proper activation functions improve learning and faster convergence.

c) Effect of vanishing and exploding gradients:

Vanishing Gradient:

- Gradients become extremely small.
- Early layers learn very slowly.
- Training may stop before reaching good accuracy.

Exploding Gradient:

- Gradients become very large.
- Weight updates become unstable.
- Loss increases instead of decreasing.

d) Techniques to fix the issue:

- Normalize input data (feature scaling).
- Use proper weight initialization:
    - Xavier initialization
    - He initialization
- Select suitable activation functions (ReLU, Leaky ReLU).
- Reduce learning rate if loss increases.
- Use Batch Normalization.
- Apply gradient clipping to control large gradients.

Conclusion:

Increasing loss during backpropagation is caused by gradient calculation errors, unsuitable activation functions, or unstable gradients. Proper initialization, normalization, and gradient control help the model converge.

4. Gradient Descent vs SGD Error Investigation

A model trained using Gradient Descent converges very slowly, while the same model with Stochastic Gradient Descent shows unstable loss fluctuations.

Tasks:

a) Explain why batch gradient descent is slow in large datasets.

b) Analyze the cause of fluctuations in SGD.

c) Compare the error behavior of Mini-Batch Gradient Descent.

d) Recommend the best approach and justify with error trade-offs.

Answer:

a) Why Batch Gradient Descent is slow for large datasets:

- It calculates the gradient using the entire dataset before updating weights.
- Large datasets require more computation for each update.
- Weight updates happen less frequently, making training slower.
- Memory usage increases for storing and processing all samples.

b) Cause of fluctuations in SGD:

- SGD updates weights using only one training sample at a time.
- Each sample produces a different gradient direction.
- Noise in individual samples causes loss values to increase and decrease.
- High learning rate can make fluctuations more severe.

c) Error behavior of Mini-Batch Gradient Descent:

- Uses a small group of samples for each update.
- Provides a balance between Batch GD and SGD.
- Loss is more stable than SGD.
- Faster than Batch GD because updates happen more frequently.
- Reduces noise while maintaining good learning speed.

d) Best approach and error trade-offs:

Recommended: Mini-Batch Gradient Descent

Reasons:

- Faster training than Batch Gradient Descent.
- More stable convergence than SGD.
- Requires less memory.
- Provides a good balance between speed and accuracy.

Method	Error Behavior	Advantage	Limitation
Batch GD	Smooth but slow decrease	Stable updates	Slow for large data
SGD	High fluctuations	Fast learning	Unstable loss
Mini-Batch GD	Moderate fluctuations	Fast and stable	Requires batch size tuning

Conclusion:

Mini-Batch Gradient Descent is preferred because it reduces SGD's instability while avoiding the slow computation of Batch Gradient Descent.

5. Curse of Dimensionality Error Analysis

A machine learning model built using high-dimensional data performs well on training data but poorly on test data due to overfitting, related to the Curse of Dimensionality.

Tasks:

a) Explain how high dimensionality increases model error.

b) Identify signs of overfitting in this scenario.

c) Analyze why distance-based learning fails in high dimensions.

d) Suggest dimensionality reduction or regularization techniques to improve performance.

Answer:

a) How high dimensionality increases model error

- Increasing the number of features makes the data space larger.
- Data points become sparse, making it difficult for the model to learn patterns.
- The model may learn noise instead of useful information.
- This increases generalization error and causes overfitting.

b) Signs of overfitting

- Training accuracy is very high.
- Test accuracy is significantly lower.
- Training loss decreases but validation loss increases.
- Model performs poorly on new/unseen data.
- Model becomes too complex compared to available data.

c) Why distance-based learning fails in high dimensions

- In high-dimensional spaces, distances between data points become very similar.
- Nearest and farthest points appear almost equally distant.
- Algorithms like KNN struggle to identify meaningful neighbors.
- Irrelevant features increase noise and reduce accuracy.

d) Techniques to improve performance

Dimensionality Reduction:

- Principal Component Analysis (PCA) to reduce unnecessary features.
- Feature selection to keep only important variables.
- Autoencoders for nonlinear dimensionality reduction.

Regularization:

- L1 Regularization (Lasso) removes less important features.
- L2 Regularization (Ridge) reduces large weights.
- Dropout in neural networks prevents over-dependence on features.

Conclusion:

High-dimensional data increases complexity and causes overfitting. Feature reduction, regularization, and proper preprocessing help improve model generalization and reduce error.